

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)

Assessment Tool Weightage: 40%

Dr. Dumpala Prasanth

QUESTIONS: 10

Total Marks: 50

Annexure A

Descriptive Experiment Exam

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Name : PC Deekshitha

Reg No : 192472175

Course code : MLA0304

Course Name : Reinforcement Learning

Experiment 1

Aim: To install and configure Python, TensorFlow, Keras, Gymnasium, NumPy, and Matplotlib libraries and verify the Reinforcement Learning environment.

```
import numpy as np
import matplotlib
import gymnasium as gym
import tensorflow as tf
import keras

print("NumPy Version:", np.__version__)
print("Matplotlib Version:", matplotlib.__version__)
print("Gymnasium Version:", gym.__version__)
print("TensorFlow Version:", tf.__version__)
print("Keras Version:", keras.__version__)

env = gym.make("CartPole-v1")
observation, info = env.reset()

print("\nEnvironment Loaded Successfully")
print("Initial Observation:")
print(observation)

env.close()
```

```
NumPy Version: 2.0.2
Matplotlib Version: 3.10.0
Gymnasium Version: 1.3.0
TensorFlow Version: 2.20.0
Keras Version: 3.13.2
```

```
Environment Loaded Successfully
Initial Observation:
[-0.04085411 -0.03178108 -0.01451789 -0.03079367]
```

Experiment 2

Aim: To create and interact with a Gymnasium environment to understand states, actions, rewards, and episodes.

```

▶ import gymnasium as gym

env = gym.make("FrozenLake-v1")

state, info = env.reset()

print("Initial State:", state)

for i in range(10):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print("Step:", i + 1)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward)

    if terminated or truncated:
        print("Episode Finished")
        break

env.close()

```

```

Initial State: 0
Step: 1
Action: 1
Next State: 0
Reward: 0
Step: 2
Action: 1
Next State: 0
Reward: 0
Step: 3
Action: 2
Next State: 1
Reward: 0
Step: 4
Action: 3
Next State: 2
Reward: 0
Step: 5
Action: 0
Next State: 2
Reward: 0
Step: 6
Action: 2
Next State: 2
Reward: 0
Step: 7
Action: 1
Next State: 3
Reward: 0
Step: 8
Action: 2
Next State: 7

```

Experiment 3

Aim: To implement the basic Reinforcement Learning framework consisting of an agent, environment, actions, states, rewards, and policy.



```
import gymnasium as gym

env = gym.make("FrozenLake-v1")

state, info = env.reset()

total_reward = 0

for step in range(20):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    total_reward += reward

    print("Current State:", state)
    print("Action:", action)
    print("Reward:", reward)

    state = next_state

    if terminated or truncated:
        break

print("Total Reward:", total_reward)

env.close()
```

```
... Current State: 0
     Action: 2
     Reward: 0
     Current State: 1
     Action: 1
     Reward: 0
     Total Reward: 0
```

Experiment 4

Aim: To simulate state transitions and reward functions in a simple Markov Decision Process.

```
import random

states = ["A", "B", "C"]

rewards = {
    "A": 5,
    "B": 2,
    "C": 10
}

current_state = "A"

for i in range(10):

    print("Current State:", current_state)
    print("Reward:", rewards[current_state])

    current_state = random.choice(states)
```

```
... Current State: A
    Reward: 5
    Current State: C
    Reward: 10
    Current State: B
    Reward: 2
    Current State: B
    Reward: 2
    Current State: B
    Reward: 2
    Current State: C
    Reward: 10
    Current State: A
    Reward: 5
    Current State: B
    Reward: 2
    Current State: C
    Reward: 10
    Current State: C
    Reward: 10
```

Experiment 5

Aim: To calculate state values using the Bellman Expectation Equation.

```
▶ gamma = 0.9

reward = [5, 2, 10]

value = [0, 0, 0]

for iteration in range(10):

    new_value = []

    for r in reward:

        v = r + gamma * max(value)

        new_value.append(v)

    value = new_value

    print("Iteration", iteration + 1)
    print(value)
```

```
... Iteration 1
[5.0, 2.0, 10.0]
Iteration 2
[14.0, 11.0, 19.0]
Iteration 3
[22.1, 19.1, 27.1]
Iteration 4
[29.39, 26.39, 34.39]
Iteration 5
[35.951, 32.951, 40.951]
Iteration 6
[41.8559, 38.8559, 46.8559]
Iteration 7
[47.17031, 44.17031, 52.17031]
Iteration 8
[51.953279, 48.953279, 56.953279]
Iteration 9
[56.2579511, 53.2579511, 61.2579511]
Iteration 10
[60.13215599, 57.13215599, 65.13215599]
```

Experiment 6

Aim: To implement Random, Greedy, and ϵ -Greedy action-selection strategies.

```
▶ import random

values = [10, 20, 30, 15]

epsilon = 0.2

for i in range(20):

    if random.random() < epsilon:
        action = random.randint(0, 3)
        print("Exploration Action:", action)
    else:
        action = values.index(max(values))
        print("Exploitation Action:", action)
```

```
... Exploration Action: 0
    Exploration Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploration Action: 0
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploitation Action: 2
    Exploration Action: 3
```

Experiment 7

Aim: To implement the Multi-Armed Bandit problem using the ϵ -Greedy algorithm.

```
▶ import numpy as np

arms = 5

counts = np.zeros(arms)

values = np.zeros(arms)

for t in range(100):

    if np.random.rand() < 0.1:
        arm = np.random.randint(arms)
    else:
        arm = np.argmax(values)

    reward = np.random.randint(1, 11)

    counts[arm] += 1

    values[arm] += (reward - values[arm]) / counts[arm]

print("Number of Selections:")
print(counts)

print("Estimated Rewards:")
print(values)
```

```
... Number of Selections:
[32. 52.  0. 16.  0.]
Estimated Rewards:
[5.5          6.51923077 0.          5.4375      0.          ]
```

Experiment 8

Aim: To visualize cumulative rewards obtained by an RL agent using Matplotlib.

```
import matplotlib.pyplot as plt
import random

episodes = 100

rewards = []

total = 0

for i in range(episodes):

    total += random.randint(0, 10)

    rewards.append(total)

plt.plot(rewards)

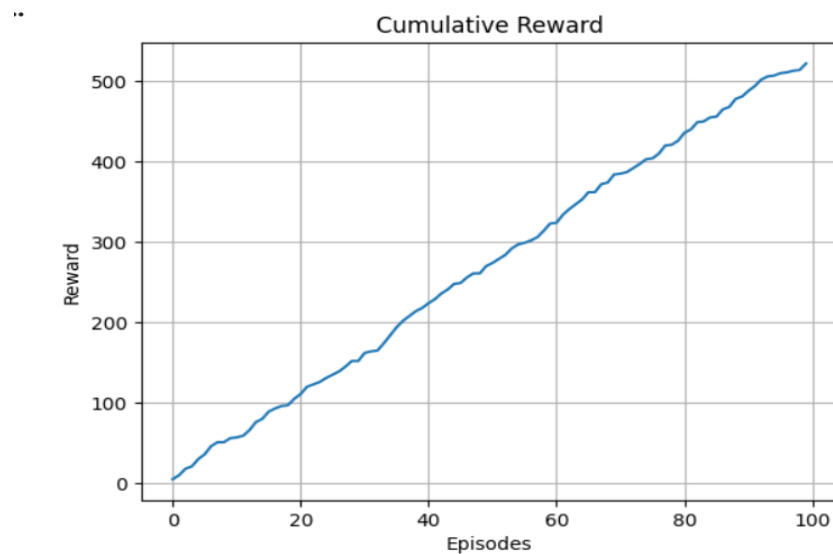
plt.title("Cumulative Reward")

plt.xlabel("Episodes")

plt.ylabel("Reward")

plt.grid(True)

plt.show()
```



Experiment 9

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for Reinforcement Learning.

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(4,)),
    tf.keras.layers.Dense(24, activation="relu"),
    tf.keras.layers.Dense(24, activation="relu"),
    tf.keras.layers.Dense(2)
])

model.compile(
    optimizer="adam",
    loss="mse"
)

model.summary()
```

... Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 24)	120
dense_1 (Dense)	(None, 24)	600
dense_2 (Dense)	(None, 2)	50

Total params: 770 (3.01 KB)
Trainable params: 770 (3.01 KB)
Non-trainable params: 0 (0.00 B)

Experiment 10

Aim: To develop a simple Reinforcement Learning agent for solving the CartPole environment and evaluate its performance.

```
import gymnasium as gym

env = gym.make("CartPole-v1")

episodes = 10

for episode in range(episodes):
    state, info = env.reset()

    total_reward = 0

    done = False

    while not done:
        action = env.action_space.sample()

        state, reward, terminated, truncated, info = env.step(action)

        total_reward += reward

        done = terminated or truncated

    print("Episode", episode + 1, "Reward:", total_reward)

env.close()
```

```
Episode 1 Reward: 48.0
., Episode 2 Reward: 21.0
Episode 3 Reward: 42.0
Episode 4 Reward: 15.0
Episode 5 Reward: 26.0
Episode 6 Reward: 19.0
Episode 7 Reward: 35.0
Episode 8 Reward: 20.0
Episode 9 Reward: 15.0
Episode 10 Reward: 17.0
```